

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет прикладной информатики и компьютерных технологий
Кафедра информационных технологий

КУРСОВАЯ РАБОТА

по дисциплине «Проектирование информационных систем»

на тему:

**«Разработка системы автоматизированного сбора и первичной
обработки данных из внешнего API с использованием Apache Airflow и
объектного хранилища»**

Выполнил:

Руководитель:

Группа:

Дата защиты:

студент

Л. Судиловский

«___» _____ 2026

г.

Минск, 2026

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Тема работы: «Разработка системы автоматизированного сбора и первичной обработки данных из внешнего API с использованием Apache Airflow и объектного хранилища».

Исходные данные: репозиторий `course_project`, учебный проект `pet_project_earthquake`, документация Apache Airflow, DuckDB, MinIO и OpenSky Network API, а также архитектурная схема ETL-конвейера.

Цель работы: разработать и описать систему, которая по расписанию получает данные из внешнего API, выполняет первичную нормализацию результата и сохраняет данные в S3-совместимое объектное хранилище в формате Parquet.

Основные задачи:

- проанализировать предметную область автоматизированного сбора данных и особенности внешнего API;
- обосновать выбор Apache Airflow, DuckDB, MinIO, Docker Compose, PostgreSQL и Redis;
- спроектировать архитектуру ETL-конвейера и описать протоколы взаимодействия компонентов;
- реализовать DAG Airflow для загрузки данных OpenSky Network API в объектное хранилище;
- описать вопросы конфигурации, безопасности, идиempотентности, качества и эксплуатации решения.

Ожидаемый результат: работоспособный локальный прототип data engineering системы, в котором Apache Airflow управляет запуском задач, DuckDB читает JSON из внешнего API и записывает Parquet-файлы в S3-совместимое хранилище MinIO.

ОГЛАВЛЕНИЕ

Оглавление обновляется автоматически в Microsoft Word/LibreOffice

ВВЕДЕНИЕ

Современные информационные системы всё чаще строятся вокруг регулярного обмена данными между внешними сервисами, внутренними аналитическими платформами и хранилищами. Для организаций, которые принимают решения на основе данных, важно не только получить сведения из внешнего API, но и сделать этот процесс повторяемым, контролируемым и воспроизводимым. Ручная загрузка файлов или разовые скрипты быстро становятся источником ошибок: сложно гарантировать периодичность запуска, трудно отследить сбои, а результаты разных запусков могут храниться в несогласованном виде.

Тема данной курсовой работы посвящена разработке системы автоматизированного сбора и первичной обработки данных из внешнего API с использованием Apache Airflow и объектного хранилища. Практическая часть реализована на основе учебного data engineering проекта, однако область решения намеренно сужена до одного завершённого этапа — загрузки сырых данных из OpenSky Network API в S3-совместимое хранилище MinIO. Такой подход позволяет сосредоточиться на наиболее важной для дальнейшей аналитики части конвейера: надёжном извлечении данных и сохранении исходного слоя в формате, пригодном для последующей обработки.

Актуальность работы определяется ростом роли автоматизированных ETL/ELT-процессов. Данные из внешних API могут изменяться каждую минуту, иметь ограничения по доступности, задержки ответа и нестабильную структуру. Поэтому система сбора должна включать планировщик, механизм повторных попыток, изолированную инфраструктуру, единый формат хранения и безопасное управление ключами доступа. В проекте эти задачи решаются средствами Apache Airflow, Docker Compose, DuckDB и MinIO.

Объектом исследования является процесс автоматизированного сбора данных из внешнего REST API. Предметом исследования является

программная и инфраструктурная реализация конвейера, который получает данные OpenSky Network API, выполняет первичное преобразование JSON-ответа и сохраняет результат в объектном хранилище в формате Parquet.

Цель курсовой работы — разработать и описать систему автоматизированного сбора и первичной обработки данных из внешнего API с использованием Apache Airflow и S3-совместимого объектного хранилища. Для достижения цели необходимо решить следующие задачи: изучить предметную область и требования к сбору данных; выбрать технологический стек; спроектировать архитектуру взаимодействия компонентов; реализовать DAG Airflow; описать хранение, безопасность, качество данных и эксплуатацию решения; оценить результат и направления развития проекта.

Практическая значимость работы заключается в том, что полученный прототип может быть использован как основа для дальнейшего развития учебного или прикладного data engineering проекта. Слой raw/bronze, сформированный в объектном хранилище, может стать источником для последующих этапов: загрузки в реляционную базу данных, построения витрин, проверки качества и визуализации.

1 Теоретические основы автоматизированного сбора данных

1.1 Понятие автоматизированного сбора данных

Автоматизированный сбор данных представляет собой процесс регулярного получения информации из внешних или внутренних источников без участия пользователя в каждом отдельном запуске. В отличие от ручной выгрузки, автоматизированный подход предполагает наличие расписания, единых правил обработки, журналирования и механизма реакции на ошибки. В data engineering такой процесс обычно рассматривается как часть ETL- или ELT-конвейера, где Extract отвечает за извлечение, Transform — за преобразование, а Load — за загрузку в целевое хранилище.

В рассматриваемом проекте реализована часть Extract и первичного Load. Система обращается к OpenSky Network API, получает актуальные

сведения о воздушных судах, разворачивает вложенную структуру ответа и сохраняет результат в объектное хранилище. Преобразования на данном этапе минимальны: данные сохраняются максимально близко к исходному виду. Это соответствует практике построения lakehouse-архитектур, где raw- или bronze-слой содержит исходные сведения и служит неизменяемой точкой восстановления для будущих обработок.

Ключевые требования к такой системе включают регулярность, воспроизводимость, наблюдаемость и устойчивость к ошибкам. Регулярность достигается расписанием DAG. Воспроизводимость обеспечивается контейнерной инфраструктурой и фиксированными версиями зависимостей. Наблюдаемость реализуется средствами веб-интерфейса Airflow и журналами выполнения задач. Устойчивость повышается за счёт retry-механизма, ограничения параллельности и разнесения компонентов по сервисам Docker Compose.

1.2 Внешний API как источник данных

Внешний API является интерфейсом, через который сторонний сервис предоставляет данные другим приложениям. OpenSky Network API публикует информацию о воздушном пространстве и состоянии воздушных судов. Согласно документации OpenSky, данные представляются в виде state vectors — векторов состояния воздушных судов, содержащих идентификатор, временные метки, координаты, скорость, курс и другие параметры. Ответ метода `/api/states/all` возвращается в формате JSON, что удобно для первичного получения, но требует нормализации перед долгосрочным хранением.

Особенность такого источника состоит в том, что он описывает текущее состояние, а не историческую таблицу с заранее известным набором строк. Это означает, что при каждом запуске конвейер получает снимок состояния на момент обращения. Для учебного проекта такой источник хорошо демонстрирует типичную задачу data engineering: внешний сервис

живёт независимо от локальной системы, а значит, конвейер должен учитывать сетевое взаимодействие, возможные задержки, ограничения API и необходимость сохранять результат запуска с привязкой к дате интервала.

Формат JSON удобен для обмена между сервисами, но для аналитического хранения часто менее эффективен, чем колоночные форматы. Поэтому в проекте после чтения JSON используется запись в Parquet. Parquet обеспечивает компактное хранение и эффективное чтение колонок при последующей аналитической обработке. Сохранение результата в S3-совместимое хранилище делает данные доступными для разных инструментов: DuckDB, Spark, Trino, Python-скриптов и BI-систем.

1.3 Apache Airflow как оркестратор

Apache Airflow — платформа для программного описания, планирования и мониторинга рабочих процессов. Основным объектом Airflow — DAG, то есть направленный ациклический граф задач. DAG задаёт расписание, набор задач и зависимости между ними. Каждая задача выполняет отдельную единицу работы, а Airflow отвечает за запуск по расписанию, хранение статусов, повторные попытки и предоставление интерфейса мониторинга.

В данной работе Airflow выбран потому, что он хорошо подходит для регулярных data engineering процессов. DAG `raw_from_api_to_s3` содержит три логических шага: начальный маркер `start`, задачу `get_and_transfer_api_data_to_s3` и конечный маркер `end`. Центральная задача реализована через `PythonOperator`, что позволяет использовать Python-функцию для запуска SQL-команд DuckDB. Такой подход сочетает гибкость Python и эффективность DuckDB при работе с внешними файлами и объектным хранилищем.

Для локального запуска используется `CeleryExecutor`. Он требует брокера сообщений Redis и базы метаданных PostgreSQL. PostgreSQL хранит информацию о DAG, задачах, запусках и пользователях, а Redis используется

как очередь задач между планировщиком и worker-процессами. Несмотря на то что для учебного проекта можно было бы использовать более простой executor, CeleryExecutor демонстрирует архитектуру, близкую к промышленным установкам Airflow, где задачи могут распределяться между несколькими worker-узлами.

1.4 Объектное хранилище и формат Parquet

Объектное хранилище отличается от файловой системы и реляционной базы тем, что данные размещаются в виде объектов внутри бакетов. Каждый объект имеет ключ, содержимое и метаданные. S3-совместимый интерфейс стал фактическим стандартом для data lake решений, поскольку позволяет хранить большие объёмы файлов и подключать к ним разные вычислительные инструменты. В проекте роль такого хранилища выполняет MinIO — локальное S3-совместимое решение, разворачиваемое в контейнере.

В MinIO создаётся bucket bronze, который соответствует первичному слою хранилища. Внутри бакета данные организуются по пути raw/opensky/<дата>/<файл>. Такая структура облегчает навигацию, позволяет отделить источник от слоя обработки и создаёт основу для партиционирования по дате. Даже если в дальнейшем появятся другие источники, их можно будет разместить рядом, сохранив единый принцип каталогизации.

Формат Parquet выбран как колоночный формат хранения. Он удобен для аналитических нагрузок, потому что позволяет читать только нужные столбцы, поддерживает сжатие и хорошо интегрируется с современными инструментами обработки данных. В рассматриваемом DAG запись в Parquet выполняется командой DuckDB COPY TO, направленной в S3-путь. DuckDB через расширение httpfs выступает как лёгкий движок первичной обработки: он читает JSON из API, преобразует вложенный массив states в строки и записывает результат в объектное хранилище.

1.5 ETL, ELT и место raw-слоя в архитектуре

В классическом подходе ETL данные сначала извлекаются из источника, затем преобразуются и после этого загружаются в целевую систему. В ELT данные сначала помещаются в хранилище, а преобразования выполняются уже внутри аналитической платформы. Для современных data lake и lakehouse решений чаще используется логика ELT, поскольку объектное хранилище позволяет недорого сохранять исходные данные и возвращаться к ним при изменении требований.

Рассматриваемая система ближе к ELT-подходу. DAG выполняет извлечение из внешнего API и минимальную техническую нормализацию, необходимую для сохранения результата в Parquet. Бизнес-правила, агрегации и витрины намеренно не реализуются на этом этапе. Это снижает риск потери исходной информации: если в будущем изменится логика расчётов, можно перечитать raw-слой и построить новые производные наборы данных.

Raw-слой выполняет роль доверенного архива поступивших данных. Он должен быть организован так, чтобы можно было понять источник, дату получения и формат файла. Поэтому в проекте путь содержит слой raw, название источника opensky и дату интервала. Такая схема каталогов проста, но уже соответствует распространённой практике построения зон данных: bronze/raw для исходных файлов, silver/ods для очищенных и типизированных данных, gold/dm для витрин и показателей.

Даже если курсовой проект ограничивается первым этапом, выбранная структура не закрывает путь к развитию. Добавление новых DAG может происходить без изменения уже сохранённых объектов. Например, отдельный DAG может читать s3://bronze/raw/opensky, приводить массив state к именованным колонкам и сохранять результат в silver-слой. Другой DAG может строить агрегаты по странам происхождения, высотам, скоростям или географическим областям. Таким образом, корректная организация raw-слоя является фундаментом всей последующей аналитики.

2 Проектирование системы автоматизированной загрузки данных

2.1 Общая архитектура решения

Архитектура разработанного решения представляет собой локальный ETL-конвейер, развернутый с помощью Docker Compose. В центре архитектуры находится Apache Airflow, который управляет расписанием и выполнением DAG. Внешним источником данных является OpenSky Network API, доступный по HTTPS. Целевым хранилищем выступает MinIO, предоставляющий S3 API поверх HTTP. Для внутренних нужд Airflow используются PostgreSQL и Redis.

Пользователь взаимодействует с системой через два веб-интерфейса. Airflow UI доступен на порту 8080 и используется для просмотра DAG, запусков, логов и статусов задач. MinIO Console доступна на порту 9001 и позволяет просматривать бакеты и сохранённые объекты. Непосредственная запись данных выполняется не через браузер, а worker-компонентом Airflow, внутри которого запускается PythonOperator и DuckDB.

Главная особенность архитектуры состоит в разделении ролей. Airflow отвечает за оркестрацию, DuckDB — за чтение и запись данных, MinIO — за хранение, PostgreSQL — за метаданные Airflow, Redis — за распределение задач Celery. Такое разделение делает систему понятной и расширяемой: каждый компонент можно заменить или масштабировать независимо, не меняя концепцию конвейера.

Архитектура ETL-конвейера и протоколы взаимодействия

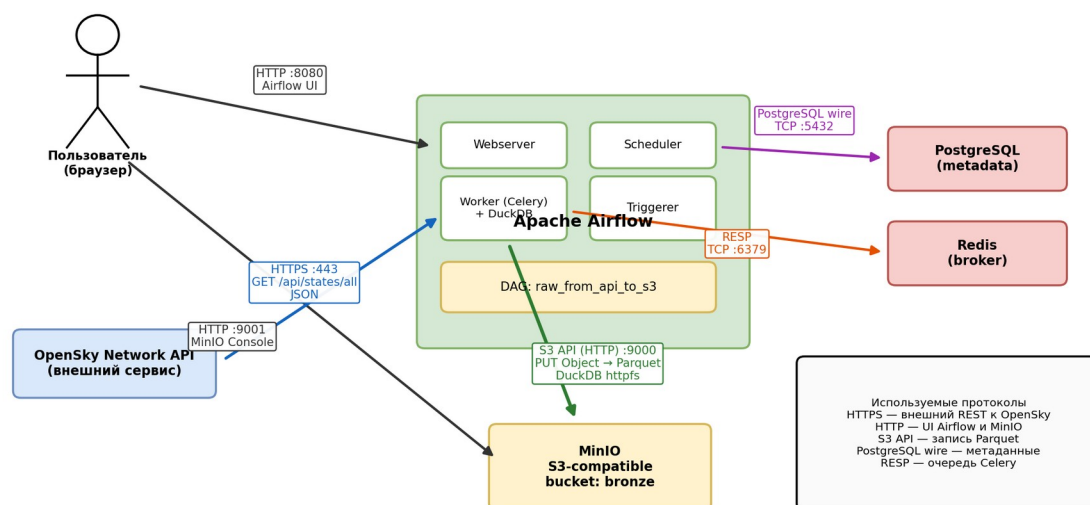


Рисунок 1 – Архитектура ETL-конвейера и протоколы взаимодействия компонентов

2.2 Состав Docker-инфраструктуры

Файл `docker-compose.yaml` основан на официальном шаблоне Apache Airflow 2.10.5 для CeleryExecutor. В проекте определены сервисы `minio`, `postgres`, `redis`, `airflow-webserver`, `airflow-scheduler`, `airflow-worker`, `airflow-triggerer`, `airflow-init`, `airflow-cli` и опциональный `flower`. Также в исходном учебном проекте присутствовали PostgreSQL для DWH и Metabase, однако в курсовой реализации они не используются и закомментированы, поскольку тема ограничена автоматической загрузкой данных в S3.

Сервис `minio` использует образ `minio/minio` и публикует порты 9000 и 9001. Порт 9000 предназначен для S3 API, а 9001 — для консоли управления. Для учебного запуска в `docker-compose` указаны локальные демонстрационные учётные данные `minioadmin/minioadmin`. Доступ DAG к S3 не зашивается в код, а берётся из Variables Airflow: `access_key` и `secret_key`. В курсовой работе реальные значения ключей не раскрываются, так как они относятся к конфигурации окружения.

Сервис `postgres` хранит метаданные Airflow и поднимается на базе PostgreSQL 13. Redis 7.2-bookworm используется как брокер Celery. Airflow

webserver предоставляет пользовательский интерфейс; scheduler анализирует DAG и создаёт запуски; worker выполняет задачи; triggerer предназначен для асинхронных операций и входит в стандартную инфраструктуру Airflow. airflow-init выполняет миграции базы и создаёт начального пользователя веб-интерфейса.

Таблица 1 – Компоненты программной инфраструктуры

Компонент	Назначение	Порт/ протокол	Роль в проекте
Apache Airflow Webserver	Веб-интерфейс управления DAG	HTTP 8080	Просмотр запусков, логов и статусов
Apache Airflow Scheduler	Планирование запусков	PostgreSQL wire	Создаёт DAG run по расписанию
Apache Airflow Worker	Выполнение задач	RESP, HTTPS, S3 API	Запускает PythonOperator и DuckDB
PostgreSQL	База метаданных Airflow	TCP 5432	Хранит статусы, DAG run, пользователей
Redis	Брокер Celery	TCP 6379 / RESP	Передаёт задачи worker-процессам
MinIO	S3-совместимое объектное хранилище	HTTP 9000/9001	Хранит Parquet-файлы в bucket bronze
DuckDB	Встроенный аналитический движок	httpfs/S3	Читает API и пишет Parquet

2.3 Потоки данных и протоколы взаимодействия

Поток данных начинается с запуска DAG по расписанию. Airflow Scheduler создаёт экземпляр DAG run и помещает задачу в очередь Celery. Worker получает задачу через Redis, исполняет Python-код и запускает SQL-команды DuckDB. DuckDB обращается к OpenSky Network API по HTTPS на порт 443, получает JSON-ответ метода /api/states/all и разворачивает массив states с помощью функции unnest.

После чтения API DuckDB через расширение httpfs подключается к S3-совместимому endpoint minio:9000. Параметр s3_url_style устанавливается в path, что подходит для локального MinIO. Также указываются access key, secret key и отключение SSL, поскольку взаимодействие между

контейнерами происходит внутри локальной Docker-сети по HTTP. Результат сохраняется командой COPY TO в Parquet-файл внутри бакета bronze.

Отдельный поток технических данных связан с работой самого Airflow. Scheduler и webserver обращаются к PostgreSQL по протоколу PostgreSQL wire на порт 5432, чтобы записывать и читать метаданные. Worker и scheduler взаимодействуют с Redis по протоколу RESP на порт 6379. Пользовательские интерфейсы доступны по HTTP: Airflow UI на 8080 и MinIO Console на 9001. Таким образом, архитектура содержит как внешние протоколы обмена данными, так и внутренние протоколы управления выполнением задач.

2.4 Логическая структура DAG

DAG `raw_from_api_to_s3` описан в файле `dags/raw_from_api_to_S3.py`. Его владелец указан как `L.sudilovski`, идентификатор DAG — `raw_from_api_to_s3`, слой данных — `raw`, источник — `opensky`. В `default_args` настроены дата начала 1 мая 2026 года, `catchup`, три повторные попытки и задержка между повторами в один час. Расписание задано выражением `0 5 * * *`, то есть запуск выполняется ежедневно в 05:00.

Функция `get_dates` извлекает из контекста Airflow начало и конец интервала данных. Это важно, потому что Airflow работает не просто с текущей датой запуска, а с логическим интервалом обработки. В рассматриваемом DAG `start_date` используется в имени каталога и файла, благодаря чему результат каждого запуска получает отдельный путь в объектном хранилище. Такой подход облегчает проверку истории запусков и предотвращает смешивание разных снимков данных.

Центральная функция `get_and_transfer_api_data_to_s3` открывает соединение DuckDB, настраивает расширение `httpfs`, указывает параметры S3 и выполняет COPY-запрос. Внутри запроса данные читаются из `read_json_auto('https://opensky-network.org/api/states/all')`, поле `states` разворачивается в строки, после чего результат записывается в `s3://bronze/raw/opensky/<дата>/<файл>`. Конструкция `start >>`

`get_and_transfer_api_data_to_s3 >> end` делает зависимости явными и удобными для отображения в Airflow UI.

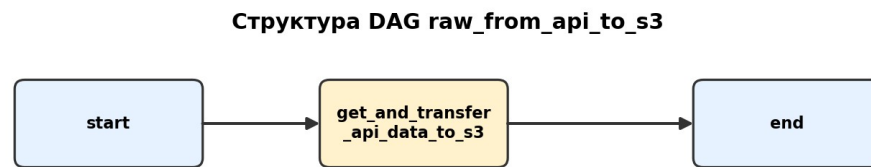


Рисунок 2 – Последовательность задач DAG raw_from_api_to_s3

2.5 Структура данных и правила именования объектов

OpenSky Network API возвращает объект JSON, содержащий время формирования ответа и массив `states`. Каждый элемент массива представляет собой `state vector` — упорядоченный список значений, относящихся к одному воздушному судну. В документации OpenSky описываются такие смысловые поля, как ICAO24-идентификатор, позывной, страна происхождения, временные метки, координаты, высота, признак нахождения на земле, скорость, курс, вертикальная скорость и другие параметры.

В текущей реализации массив `states` разворачивается в столбец `state` без детального именования каждого элемента. Для raw-слоя это допустимо, поскольку основная задача — сохранить исходный снимок без потери данных. Однако уже на уровне проектирования важно понимать, что следующий слой должен выполнить семантическую расшифровку массива. Это позволит заменить позиционный доступ к элементам массива на именованные колонки, упростить проверки качества и сделать данные пригодными для SQL-аналитики.

Правила именования объектов в S3 должны быть стабильными. В проекте используется шаблон `s3://bronze/raw/opensky/YYYY-MM-DD/YYYY-MM-DD_00-00-00.parquet`. Он содержит `bucket`, слой, источник, партицию по дате и имя файла. Такой шаблон полезен не только человеку, который

открывает MinIO Console, но и программным инструментам: они могут перечислять каталоги по дате и обрабатывать только нужные интервалы.

При промышленном развитии системы можно добавить дополнительные элементы пути: час запуска, регион данных, версию схемы или идентификатор загрузки. В учебном проекте это избыточно, потому что DAG выполняется один раз в день и сохраняет один снимок. Тем не менее выбранная структура совместима с будущим расширением и не требует изменения базового принципа хранения.

3 Реализация программного решения

3.1 Реализация получения и первичной обработки данных

Практическая реализация построена как минимальный, но завершённый DAG. В начале файла импортируются logging, duckdb, pendulum, классы DAG, Variable, EmptyOperator и PythonOperator. Использование Variable позволяет хранить ключи доступа к S3 в Airflow, а не в исходном коде. Это особенно важно, поскольку часть локальных файлов и секретов исключена из Git с помощью .gitignore: журналы, данные, .env, виртуальные окружения и файл cred.py не попадают в репозиторий.

Первичная обработка данных в данном проекте заключается не в бизнес-трансформации, а в техническом приведении ответа API к файловому аналитическому формату. OpenSky возвращает JSON-объект, где список состояний находится во вложенном поле states. DuckDB позволяет прочитать такой JSON напрямую и выполнить unnest(states), получив табличный результат. Сохранять исходный снимок в Parquet рационально: дальнейшие этапы смогут читать его быстрее и дешевле, чем исходный JSON.

Важным свойством решения является отсутствие промежуточных файлов на локальном диске. Данные читаются из HTTPS-источника и записываются непосредственно в MinIO через S3 API. Это уменьшает количество временных состояний и упрощает эксплуатацию. При этом

Airflow сохраняет логи выполнения, что позволяет отследить дату запуска, успешность операции и сообщения об ошибках.

Ключевой фрагмент SQL-команды DuckDB имеет следующий вид:

```
COPY (
    SELECT
        time,
        state
    FROM read_json_auto('https://opensky-network.org/api/states/all')
    TO 's3://bronze/raw/opensky/{start_date}/{start_date}_00-00-00.parquet';
```

3.2 Конфигурация доступа к объектному хранилищу

Доступ к MinIO в DAG задаётся через переменные Airflow `access_key` и `secret_key`. На уровне SQL DuckDB они передаются в параметры `s3_access_key_id` и `s3_secret_access_key`. В тексте курсовой работы реальные значения не приводятся: достаточно указать принцип хранения секретов и место их использования. Такой подход соответствует базовым требованиям безопасности и предотвращает попадание ключей в Git-репозиторий.

В локальной инфраструктуре endpoint S3 задан как `minio:9000`. Имя `minio` является DNS-именем сервиса внутри Docker Compose сети. Параметр `s3_use_ssl = FALSE` используется потому, что внутри локального стенда взаимодействие происходит по HTTP. Для промышленной эксплуатации это решение необходимо изменить: использовать TLS, отдельные сервисные учётные записи, ограниченные политики доступа к бакету и централизованное управление секретами.

Структура пути `s3://bronze/raw/opensky/<дата>/<дата>_00-00-00.parquet` отражает несколько проектных решений. Bucket `bronze` отделяет первичный слой от будущих слоёв обработки. Каталог `raw` указывает на минимальную степень преобразования. Каталог `opensky` фиксирует источник данных. Дата в пути позволяет хранить разные снимки и упрощает последующую партиционную обработку.

Таблица 2 – Функциональные и нефункциональные требования

Группа требований	Требование	Реализация
Функциональные	Получать данные из внешнего API	GET-запрос к OpenSky /api/states/all через DuckDB

Функциональные	Сохранять результат в объектное хранилище	COPY TO в s3://bronze/raw/opensky/...
Функциональные	Запускать процесс по расписанию	DAG Airflow с cron-расписанием 0 5 * * *
Надёжность	Повторять задачу при временных ошибках	retries = 3, retry_delay = 1 час
Безопасность	Не хранить ключи в Git	Airflow Variables и .gitignore
Наблюдаемость	Предоставлять логи и статусы	Airflow UI и логи задач
Расширяемость	Поддерживать дальнейшую обработку	Parquet в S3-совместимом raw/bronze-слое

3.3 Идемпотентность, расписание и повторные попытки

Идемпотентность в контексте ETL означает, что повторный запуск задачи за тот же интервал не должен приводить к неконтролируемому дублированию или противоречивому состоянию данных. В реализованном DAG результат записывается по детерминированному пути, зависящему от даты интервала. Поэтому повторный запуск за одну и ту же дату перезапишет или обновит тот же логический объект, а не создаст произвольное количество новых файлов с разными именами.

Расписание 0 5 * * * выбрано как ежедневный запуск в утреннее время. Для учебного прототипа этого достаточно, чтобы показать автоматизацию. Параметр catchup = True означает, что Airflow может создать запуски за пропущенные интервалы начиная с start_date. Это полезно при необходимости восстановить историю, но требует осторожности: при большом числе пропущенных дней система может создать много запусков подряд. Для контроля нагрузки дополнительно заданы concurrency = 1, max_active_tasks = 1 и max_active_runs = 1.

Повторные попытки настроены через retries = 3 и retry_delay = один час. Такая настройка оправдана для внешнего API: временные сетевые ошибки, ограничения или недоступность сервиса могут исчезнуть через некоторое время. Если ошибка связана с неправильным ключом доступа, отсутствующим бакетом или изменением схемы API, повторные попытки сами по себе проблему не решат, но дадут оператору понятный статус сбоя в Airflow UI.

3.4 Сравнение с исходным учебным проектом

Курсовой проект создан на основе `pet_project_earthquake`, но имеет более узкую область. В исходном проекте реализована полноценная демонстрационная аналитическая цепочка: загрузка данных землетрясений из USGS API в S3, перенос в PostgreSQL, построение витрин и визуализация в Metabase. В текущей работе сохранена идея lakehouse-подхода, Docker-инфраструктура и использование Airflow, но практическая реализация ограничена автоматической загрузкой данных OpenSky в объектное хранилище.

Такое ограничение соответствует теме курсовой работы. В ней требуется показать систему автоматизированного сбора и первичной обработки данных из внешнего API, а не строить полный DWH и BI-контур. Поэтому сервисы `postgres_dwh` и `metabase` в `docker-compose` закомментированы. Это уменьшает сложность запуска, снижает требования к ресурсам и позволяет подробнее рассмотреть именно первый этап конвейера.

Отличается и сам источник данных. Исходный проект использовал CSV-ответ USGS API с параметрами `starttime` и `endtime`. Курсовой проект использует JSON-ответ OpenSky `/api/states/all`, который представляет актуальный снимок воздушного пространства. В результате изменяется логика чтения: вместо `read_csv_auto` применяется `read_json_auto` и `unnest`. Однако общая идея остаётся прежней: Airflow планирует запуск, DuckDB выполняет чтение и запись, а S3-совместимое хранилище сохраняет результат для последующих этапов.

3.5 Порядок развёртывания и проверки результата

Развёртывание проекта начинается с подготовки локального окружения. В репозитории указан файл `req.txt` с зависимостями `apache-airflow==2.10.5` и `duckdb==1.2.2`, а основная инфраструктура описана в `docker-compose.yaml`. Для контейнерного запуска достаточно установить Docker и Docker Compose, после чего выполнить `docker-compose up -d` из

корня проекта. При первом запуске сервис airflow-init выполняет миграции базы метаданных и создаёт пользователя веб-интерфейса.

После запуска инфраструктуры необходимо открыть Airflow UI и убедиться, что DAG `raw_from_api_to_s3` обнаружен системой. Затем в Airflow Variables должны быть добавлены значения `access_key` и `secret_key`. Эти переменные используются задачей для подключения к MinIO. Если бакет `bronze` ещё не создан, его нужно создать через MinIO Console. В учебном стенде это выполняется вручную, но в дальнейшем создание бакета можно автоматизировать отдельным `init`-скриптом или инфраструктурным кодом.

Проверка результата состоит из нескольких шагов. Сначала в Airflow UI запускается DAG вручную либо ожидается запуск по расписанию. Затем проверяется, что задачи `start`, `get_and_transfer_api_data_to_s3` и `end` завершились успешно. После этого в MinIO Console открывается bucket `bronze` и проверяется наличие объекта в каталоге `raw/opensky` за соответствующую дату. Дополнительно можно прочитать Parquet-файл через DuckDB, чтобы убедиться, что файл не пустой и содержит ожидаемую структуру.

Важно, что результат проверки не должен требовать публикации секретов. В отчёте достаточно описать, что ключи настроены в Airflow Variables, а значения скрыты. Скриншоты или демонстрационные материалы при защите можно делать так, чтобы они показывали успешный статус DAG и наличие объекта в бакете, но не раскрывали значения `access key` и `secret key`.

3.6 Соответствие реализации поставленным требованиям

Разработанная система соответствует основной цели курсовой работы: она автоматизирует получение данных из внешнего API и сохраняет их в объектном хранилище. Автоматизация достигается за счёт расписания Airflow и DAG-модели. Первичная обработка выполняется DuckDB, который

преобразует JSON-ответ во внутреннее табличное представление и сохраняет результат в Parquet.

Требование воспроизводимости обеспечивается контейнерной инфраструктурой. Все основные сервисы описаны в одном `docker-compose.yaml`, а версии Airflow и DuckDB зафиксированы в `req.txt` и настройках образа. Это позволяет перенести проект на другой компьютер с установленным Docker и получить аналогичную структуру сервисов. Для учебной курсовой работы такая воспроизводимость является достаточной.

Требование безопасности выполнено на базовом уровне: секреты доступа к S3 не размещены в исходном коде и исключённые локальные файлы не добавляются в Git. Вместе с тем в разделе оценки указаны ограничения учебного стенда и меры, необходимые для промышленного использования: отдельные сервисные пользователи, минимальные права, TLS и secret backend.

Требование расширяемости также выполняется. Хотя проект реализует только загрузку в S3, структура `raw/opensky` и формат Parquet позволяют добавить последующие DAG без изменения текущего этапа. Поэтому курсовой проект можно рассматривать как первый модуль более широкой lakehouse-системы, в которой позже появятся ODS-слой, витрины и визуализация.

4 Оценка решения и направления развития

4.1 Безопасность и управление секретами

В проекте разделены исходный код и чувствительная конфигурация. Git-репозиторий содержит DAG, `docker-compose.yaml`, `requirements`-файл и README, но не содержит локальные данные, журналы, `.env` и файл `cred.py`. Такой подход отражён в `.gitignore` и позволяет не публиковать секреты или служебные артефакты. В Airflow ключи доступа к S3 хранятся как Variables и извлекаются во время выполнения DAG.

Для учебного стенда допустимы демонстрационные значения MinIO root user и password, указанные в docker-compose. Однако в реальной эксплуатации root-пользователь не должен использоваться приложениями. Следует создать отдельного пользователя или access key с минимальными правами, например только на запись в конкретный bucket и чтение нужных объектов. Также необходимо включить TLS, ограничить доступ к UI и консоли MinIO, настроить роли Airflow и хранить секреты во внешнем secret backend.

Отдельное внимание требуется к логам. В текущей реализации SQL-строка формируется с подстановкой ключей. Если такой запрос будет выведен в лог целиком, секрет может попасть в журнал. Поэтому в дальнейшем лучше использовать механизм DuckDB secrets или конфигурировать подключение таким образом, чтобы значения ключей не появлялись в текстах запросов. В рамках курсовой работы реальные ключи не приводятся и не сохраняются в документе.

4.2 Качество данных и первичные проверки

Так как курсовой проект реализует raw-слой, набор проверок качества данных минимален, но их можно формализовать. На этапе получения данных важно контролировать доступность API, наличие поля states в ответе, ненулевой объём результата и успешность записи объекта в MinIO. Эти проверки помогают отличить успешный запуск с данными от ситуации, когда API вернул пустой или некорректный ответ.

Для OpenSky часть значений может отсутствовать: не для каждого воздушного судна доступны координаты, скорость или высота. Это не является ошибкой raw-слоя, потому что система должна сохранять данные такими, какими они пришли из источника. Однако для последующих слоёв потребуются правила типизации, фильтрации и контроля полноты. Например, можно отдельно считать количество записей, долю строк без координат и время получения снимка.

Качество хранения проверяется через структуру бакета. После успешного запуска должен появиться объект по ожидаемому пути, соответствующему дате интервала. Дополнительно можно проверять размер файла, возможность прочитать Parquet обратно через DuckDB и соответствие базового набора колонок ожиданиям. В промышленной версии такие проверки можно оформить отдельными задачами Airflow или подключить специализированные инструменты data quality.

4.3 Эксплуатация, мониторинг и восстановление

Эксплуатация решения начинается с запуска `docker-compose up -d`. После инициализации пользователь открывает Airflow UI, проверяет наличие DAG `raw_from_api_to_s3` и при необходимости задаёт Variables `access_key` и `secret_key`. Затем DAG можно включить и запустить вручную либо дождаться запуска по расписанию. Результат проверяется в MinIO Console в bucket `bronze`.

Airflow предоставляет несколько уровней мониторинга. В графе DAG видно состояние задач, в списке запусков — история выполнения, в логах — технические сообщения Python-кода и ошибки. Для CeleryExecutor дополнительно может использоваться Flower, если включить соответствующий профиль Docker Compose. PostgreSQL и Redis являются внутренними компонентами, но их доступность влияет на работоспособность всей системы.

Восстановление после ошибок зависит от причины сбоя. Если недоступен внешний API, задачу можно перезапустить позже или дождаться `retry`. Если отсутствует bucket `bronze`, его нужно создать в MinIO. Если неверно заданы Variables, требуется обновить значения в Airflow. Если сбой произошёл после частичной записи объекта, повторный запуск за тот же интервал должен привести к формированию ожидаемого файла по тому же пути.

4.4 Ограничения и направления развития

Главное ограничение текущей реализации состоит в том, что она сохраняет снимок текущего состояния OpenSky, но не строит полный исторический контур обработки. Данные не загружаются в DWH, не типизируются на ODS-слое, не проходят расширенные проверки качества и не визуализируются. Эти ограничения осознанны, поскольку тема курсовой работы сфокусирована на автоматизированном сборе и первичной обработке.

Следующим этапом развития может стать добавление задачи проверки результата после записи в S3. Например, Airflow может прочитать созданный Parquet-файл и записать в лог количество строк. Затем можно добавить слой stg или ods, где массив state будет разобран на именованные поля: icao24, callsign, origin_country, time_position, last_contact, longitude, latitude, baro_altitude, on_ground, velocity, true_track и другие параметры. Это повысит пригодность данных для аналитики.

Для приближения к промышленной эксплуатации следует улучшить безопасность и надёжность: использовать отдельные сервисные ключи, secret backend, TLS, централизованный мониторинг, алерты, лимиты обращений к API и обработку ошибок схемы. Также возможно заменить локальный MinIO на облачное S3-совместимое хранилище, а Docker Compose — на Kubernetes или управляемую инфраструктуру.

4.5 Экономическая и организационная оценка решения

Выбранный технологический стек хорошо подходит для учебного проекта, потому что все основные компоненты доступны как открытое программное обеспечение и могут быть запущены локально. Apache Airflow, DuckDB, PostgreSQL, Redis и MinIO не требуют покупки лицензий для демонстрационного применения. Основные затраты связаны не с лицензированием, а с вычислительными ресурсами, временем настройки и сопровождением инфраструктуры.

Использование Docker Compose снижает организационную сложность: студенту или разработчику не нужно вручную устанавливать каждый сервис в операционную систему. Контейнеры можно остановить, пересоздать и перенести вместе с описанием инфраструктуры. Это особенно полезно для курсовой работы, где важна возможность показать результат на защите и объяснить назначение каждого компонента.

С точки зрения трудозатрат наиболее сложными являются не строки кода DAG, а правильная настройка взаимодействия сервисов. Необходимо понимать, как Airflow использует PostgreSQL и Redis, как DuckDB подключается к S3 endpoint, почему ключи должны храниться отдельно и как проверять результат в MinIO. Поэтому проект имеет образовательную ценность: он демонстрирует не только программирование на Python, но и инженерное мышление при построении data pipeline.

Если переносить решение в промышленную среду, экономическая оценка изменится. Появятся расходы на облачное хранилище, вычислительные ресурсы, мониторинг, резервное копирование и безопасность. Однако сама архитектурная идея останется применимой: оркестратор управляет процессом, вычислительный движок выполняет обработку, объектное хранилище содержит raw-данные, а метаданные и логи позволяют контролировать выполнение.

5 Обоснование выбора технологий и сравнение с аналогами

5.1 Обоснование выбора Apache Airflow

Apache Airflow выбран как основной оркестратор, потому что он позволяет описывать рабочие процессы как код на Python, запускать их по расписанию, контролировать зависимости и видеть историю выполнения в веб-интерфейсе. Для курсового проекта это особенно важно: один и тот же DAG можно показать в коде, в интерфейсе и в логах выполнения, что делает решение прозрачным для проверки и защиты.

Airflow хорошо подходит для batch-процессов загрузки данных. Он не предназначен для обработки каждого сообщения в реальном времени, но отлично решает задачу периодической загрузки снимков из API. В проекте запуск один раз в день, retry-политика, ограничения `max_active_runs` и логи задач реализуются штатными механизмами Airflow, без написания собственного планировщика.

В качестве аналога можно рассмотреть Prefect. Prefect проще для быстрого написания Python flow и имеет современный интерфейс, но в учебных data engineering проектах Airflow чаще встречается как промышленный стандарт. Другой аналог — Dagster: он сильнее ориентирован на data assets и типизированные зависимости между наборами данных, но требует больше архитектурной подготовки. Для текущей задачи Airflow выбран как более узнаваемый и достаточный инструмент. Cron и обычный Python-скрипт не выбраны, потому что они не дают удобной истории запусков, retry, UI и графа зависимостей из коробки.

5.2 Обоснование выбора MinIO и S3-совместимого хранилища

MinIO используется как локальное объектное хранилище, совместимое с S3 API. Это позволяет разрабатывать и демонстрировать проект без доступа к облачному AWS S3, но сохранять тот же принцип работы: bucket, object key, access key, secret key и операции PUT/GET. Для курсового проекта это удобно, потому что все компоненты запускаются локально через Docker Compose.

Объектное хранилище выбрано вместо реляционной базы данных для raw-слоя по нескольким причинам. Во-первых, исходные данные из API логично хранить файлами, не навязывая им заранее жёсткую схему. Во-вторых, S3-подход хорошо масштабируется и является стандартным для data lake. В-третьих, Parquet-файлы могут читаться разными аналитическими инструментами без обязательной загрузки в конкретную СУБД.

Аналогами MinIO являются AWS S3, Yandex Object Storage, Google Cloud Storage, Ceph Object Gateway и локальная файловая система. Облачные сервисы ближе к production, но требуют аккаунта, оплаты и настройки IAM. Ceph сложнее для учебного стенда. Локальная файловая система проще, но не демонстрирует S3 API и управление бакетами. Поэтому MinIO является оптимальным компромиссом между реалистичностью и простотой запуска.

5.3 Обоснование выбора DuckDB

DuckDB выбран как встроенный аналитический SQL-движок. Он запускается внутри Python-задачи и не требует отдельного сервера. Это снижает сложность инфраструктуры: для чтения JSON из API и записи Parquet в S3 не нужно поднимать Spark, Trino или отдельный ETL-сервис. При этом DuckDB поддерживает SQL, работу с JSON/CSV/Parquet и расширение `httpfs` для S3-совместимых хранилищ.

В проекте DuckDB выполняет две функции. Первая — первичное чтение ответа API с помощью `read_json_auto`. Вторая — запись результата командой `COPY TO` в S3-путь. Такой подход компактнее, чем ручное получение JSON через `requests`, преобразование в `pandas DataFrame` и сохранение через отдельную библиотеку. SQL-запрос также проще объяснить в курсовой: вход — внешний JSON, выход — Parquet-объект в `bucket bronze`.

Аналогом мог быть Apache Spark. Spark мощнее и подходит для больших распределённых наборов данных, но для одного ежедневного API-снимка он избыточен: больше контейнеров, больше памяти и сложнее настройка. `pandas` проще для начинающих, но хуже показывает data lake подход и не является SQL-движком. Trino удобен для федеративных запросов к хранилищам, но требует отдельного сервера. Поэтому DuckDB выбран как лёгкий и достаточный инструмент первичной обработки.

5.4 Обоснование выбора Docker Compose, PostgreSQL и Redis

Docker Compose выбран для воспроизводимого локального развёртывания. Он описывает все сервисы в одном YAML-файле и позволяет запускать стенд командой `docker compose up -d`. Для курсовой работы это важно, потому что проверяющий может увидеть не только код DAG, но и состав инфраструктуры: Airflow, MinIO, PostgreSQL, Redis и вспомогательные сервисы.

PostgreSQL используется как база метаданных Airflow. Это не хранилище бизнес-данных проекта, а служебная база для DAG run, task instance, пользователей, переменных и других внутренних сущностей Airflow. Выбор PostgreSQL объясняется тем, что официальный docker-compose Airflow использует его как надёжную и распространённую СУБД. SQLite не подходит для такой конфигурации, особенно при CeleryExecutor.

Redis используется как брокер сообщений CeleryExecutor. Scheduler помещает задачи в очередь, worker получает их и выполняет. Альтернативой мог быть LocalExecutor без Redis, что было бы проще для учебного стенда. Однако CeleryExecutor лучше демонстрирует распределённую архитектуру Airflow и показывает, как задачи могут выполняться отдельными worker-процессами. KubernetesExecutor не выбран, потому что требует Kubernetes-кластера и усложнил бы курсовой проект без необходимости.

6 Результаты практической проверки

6.1 Результаты развёртывания инфраструктуры

В ходе проверки локальная инфраструктура была запущена через Docker Compose. Контейнеры Airflow webserver, scheduler, worker, triggerer, PostgreSQL, Redis и MinIO поднялись и перешли в рабочее состояние. Airflow обнаружил DAG `raw_from_api_to_s3` без ошибок импорта, что подтверждает корректность размещения DAG-файла и синтаксиса Python-кода.

В Airflow были заведены Variables `access_key` и `secret_key` для подключения к MinIO. Значения переменных в отчёте не раскрываются. Bucket `bronze` был создан в MinIO, после чего отдельной проверкой подтверждена возможность записи Parquet-файла через DuckDB `httpfs` в S3-совместимое хранилище. Это подтверждает корректность связки DuckDB → MinIO и сетевого доступа по endpoint `minio:9000`.

Тестовый запуск DAG дошёл до задачи `get_and_transfer_api_data_to_s3` и выполнил стартовую часть процесса. При обращении к внешнему OpenSky API был получен таймаут HTTP HEAD, после чего Airflow перевёл задачу в состояние `up_for_retry`. Для курсовой работы это является важным эксплуатационным результатом: система не завершается бесконтрольно, а применяет настроенную `retry`-политику при временной недоступности внешнего источника.

Результаты практической проверки прототипа

1	Docker Compose поднял Airflow, PostgreSQL, Redis и MinIO	успешно
2	Airflow обнаружил DAG <code>raw_from_api_to_s3</code> без ошибок импорта	успешно
3	Airflow Variables <code>access_key</code> и <code>secret_key</code> заданы для S3-доступа	успешно
4	DuckDB записал тестовый Parquet-файл в bucket <code>bronze</code> через S3 API	успешно
5	Запуск DAG дошёл до обращения к OpenSky API и ушёл в <code>retry</code> при таймауте	ожидаемая обработка сбоя

Вывод: локальная инфраструктура и запись в S3-совместимое хранилище подтверждены;
таймаут внешнего API показал работу `retry`-механизма Airflow и отражён как эксплуатационный риск.

Рисунок 3 – Сводка результатов практической проверки прототипа

6.2 Интерпретация полученных скриншотов

На рисунке 3 показан экран Airflow с DAG `raw_from_api_to_s3`, списком задач и состояниями выполнения. Он демонстрирует, что Airflow видит DAG, отображает структуру `workflow` и фиксирует статус центральной задачи. Такое представление полезно для оператора, поскольку позволяет

быстро понять, на каком этапе возникла проблема: до запуска, во время обращения к API или при записи в хранилище.

На рисунке 4 показана логика объектного хранилища MinIO. Bucket bronze предназначен для raw-слоя, а путь raw/opensky/<date>/<file>.parquet является ожидаемым результатом успешного DAG. В отчёте также отражена проверочная запись test/test.parquet, созданная для подтверждения того, что DuckDB может записывать Parquet-файлы в MinIO через S3 API.

На рисунке 5 сведены результаты практической проверки: инфраструктура запущена, DAG импортируется, переменные доступа настроены, запись в MinIO проверена, а таймаут OpenSky обработан механизмом retry. Эти результаты показывают, что реализованный прототип соответствует теме курсовой работы и может быть доработан после восстановления доступности внешнего API или при использовании альтернативного источника данных.

Apache Airflow UI

DAGs

DAG: raw_from_api_to_s3

is_paused: False

Task	Operator	State	Description
start	EmptyOperator	success	Начальный маркер запуска
get_and_transfer_api_data_to_s3	PythonOperator	up_for_retry	Загрузка OpenSky API → MinIO/S3
end	EmptyOperator	none	Конечный маркер DAG

Результат тестового запуска

DAG обнаружен Airflow без import errors; задача стартовала и перешла в retry из-за таймаута внешнего OpenSky API.
Это подтверждает работоспособность оркестрации и демонстрирует штатную обработку временной недоступности API.
Секреты S3 настроены через Airflow Variables и не выводятся в отчёт.

Рисунок 4 – Представление DAG и статусов задач в Airflow

MinIO Console

bucket: bronze

Object browser / bronze

Object key	Type	Status	Purpose
raw/opensky/<date>/<date>_00-00-00.parquet	Parquet	ожидаемый результат	DuckDB/bronze слой
test/test.parquet	Parquet	создан проверкой S3-записи	проверка DuckDB → MinIO
logs/	directory	ignored by Git	служебные логи Airflow
data/	directory	ignored by Git	локальное хранилище MinIO

Проверка результата

Bucket bronze создан; запись Parquet через DuckDB https в MinIO проверена отдельной командой.

Боевой объект raw/opensky формируется при успешной доступности внешнего OpenSky API.

Рисунок 5 – Проверка структуры bucket bronze в MinIO

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы была разработана и описана система автоматизированного сбора и первичной обработки данных из внешнего API с использованием Apache Airflow и объектного хранилища. Практическая реализация сфокусирована на загрузке данных OpenSky Network API в S3-совместимое хранилище MinIO. Система разворачивается локально с помощью Docker Compose и включает Apache Airflow, PostgreSQL, Redis, MinIO и DuckDB.

В теоретической части рассмотрены особенности автоматизированного сбора данных, роль внешних API, назначение Apache Airflow как оркестратора и преимущества объектного хранения в формате Parquet. В проектной части описана архитектура компонентов, протоколы взаимодействия, структура Docker-инфраструктуры и логика DAG raw_from_api_to_s3. В практической части показано, как DuckDB читает JSON-ответ API, разворачивает поле states и записывает результат в bucket bronze по дате интервала.

Поставленные задачи выполнены. Проанализирован исходный учебный проект pet_project_earthquake и выделена часть, соответствующая теме

курсовой работы. Обоснован выбор технологий, описаны настройки расписания, повторных попыток, доступа к S3 и структуры хранения. Отдельно рассмотрены вопросы безопасности: реальные ключи не помещаются в Git и должны храниться в Airflow Variables или специализированном хранилище секретов.

Разработанное решение имеет учебный характер, но отражает принципы, применимые в реальных data engineering системах: разделение ролей компонентов, воспроизводимая контейнерная инфраструктура, гав-слой в объектном хранилище, наблюдаемость через Airflow и расширяемая структура путей. В дальнейшем проект может быть дополнен проверками качества, разбором state vector на именованные поля, загрузкой в DWH, построением витрин и визуализацией данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Apache Airflow Documentation. Dags [Электронный ресурс]. – Режим доступа: <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>. – Дата доступа: 20.05.2026.
2. Apache Airflow Documentation. Running Airflow in Docker [Электронный ресурс]. – Режим доступа: <https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html>. – Дата доступа: 20.05.2026.
3. DuckDB Documentation. httpfs extension and S3 API [Электронный ресурс]. – Режим доступа: https://duckdb.org/docs/stable/core_extensions/httpfs/s3api.html. – Дата доступа: 20.05.2026.
4. MinIO Documentation. Object Store [Электронный ресурс]. – Режим доступа: <https://docs.min.io/community/minio-object-store/>. – Дата доступа: 20.05.2026.

5. OpenSky Network API Documentation [Электронный ресурс]. – Режим доступа: <https://opensky-network.org/data/api-docs>. – Дата доступа: 20.05.2026.
6. Korsakov I. Pet project earthquake [Электронный ресурс]. – Режим доступа: https://github.com/k0rsakov/pet_project_earthquake. – Дата доступа: 20.05.2026.
7. Apache Software Foundation. docker-compose.yaml for Airflow 2.10.5 [Электронный ресурс]. – Режим доступа: <https://airflow.apache.org/docs/apache-airflow/2.10.5/docker-compose.yaml>. – Дата доступа: 20.05.2026.
8. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления.
9. ГОСТ 7.1–2003. Библиографическая запись. Библиографическое описание. Общие требования и правила составления.
10. Методические рекомендации по оформлению курсовых и дипломных работ / Институт бизнеса БГУ. – Минск : БГУ, 2020. – 36 с.
11. Официальный репозиторий курсового проекта course_project [Электронный ресурс]. – Режим доступа: https://github.com/SailorVAC/course_project. – Дата доступа: 20.05.2026.
12. Apache Airflow. Concepts and architecture [Электронный ресурс]. – Режим доступа: <https://airflow.apache.org/docs/>. – Дата доступа: 20.05.2026.

ПРИЛОЖЕНИЕ А

Полный листинг DAG raw_from_api_to_S3.py

```

import logging
import duckdb
import pendulum
from airflow import DAG
from airflow.models import Variable
from airflow.operators.empty import EmptyOperator
from airflow.operators.python import PythonOperator

# Конфигурация DAG
OWNER = "L.sudilovskiy"
DAG_ID = "raw_from_api_to_s3"

# Используемые таблицы в DAG
LAYER = "raw"
SOURCE = "opensky"

# S3
ACCESS_KEY = Variable.get("access_key")
SECRET_KEY = Variable.get("secret_key")

LONG_DESCRIPTION = """
# LONG DESCRIPTION
"""

SHORT_DESCRIPTION = "SHORT DESCRIPTION"

args = {
    "owner": OWNER,
    "start_date": pendulum.datetime(2026, 5, 1, tz="Europe/Moscow"),
    "catchup": True,
    "retries": 3,
    "retry_delay": pendulum.duration(hours=1),
}

def get_dates(**context) -> tuple[str, str]:
    """
    start_date = context["data_interval_start"].format("YYYY-MM-DD")
    end_date = context["data_interval_end"].format("YYYY-MM-DD")
    return start_date, end_date

def get_and_transfer_api_data_to_s3(**context):
    """
    start_date, end_date = get_dates(**context)
    logging.info(f"Start load for dates: {start_date}/{end_date}")
    con = duckdb.connect()

    con.sql(
        f"""
        SET TIMEZONE='UTC';
        INSTALL httpfs;
        LOAD httpfs;
        SET s3_url_style = 'path';
        SET s3_endpoint = 'minio:9000';
        SET s3_access_key_id = '{ACCESS_KEY}';
        SET s3_secret_access_key = '{SECRET_KEY}';
        SET s3_use_ssl = FALSE;

        COPY
        (
        SELECT
    """

```

```

                                time,
                                unnest(states) as state
                                FROM
                                read_json_auto('https://opensky-network.org/api/states/all')
) TO 's3://bronze/{LAYER}/{SOURCE}/{start_date}/{start_date}_00-00-
00.parquet';

                                """
                                )

                                con.close()
                                logging.info(f"Download for date {start_date} success: {start_date}")

with
                                DAG(
                                dag_id=DAG_ID,
                                schedule_interval="@
                                5 *
                                *",
                                default_args=args,
                                tags=["s3", "raw"],
                                description=SHORT_DESCRIPTION,
                                concurrency=1,
                                max_active_tasks=1,
                                max_active_runs=1,
                                )
                                as
                                dag:
                                dag.doc_md = LONG_DESCRIPTION

                                start = EmptyOperator(
                                task_id="start",
                                )

                                get_and_transfer_api_data_to_s3 = PythonOperator(
                                task_id="get_and_transfer_api_data_to_s3",
                                python_callable=get_and_transfer_api_data_to_s3,
                                )

                                end = EmptyOperator(
                                task_id="end",
                                )

                                start >> get_and_transfer_api_data_to_s3 >> end

```

ПРИЛОЖЕНИЕ Б

Полный листинг docker-compose.yaml

В листинге показана инфраструктура проекта: Airflow, PostgreSQL, Redis и MinIO. Закомментированные сервисы DWH/BI оставлены как направление развития, но не используются в рамках темы курсовой.

```
# Licensed to the Apache Software Foundation (ASF) under one
# or more contributor license agreements. See the NOTICE file
# distributed with this work for additional information
# regarding copyright ownership. The ASF licenses this file
# to you under the Apache License, Version 2.0 (the
# "License"); you may not use this file except in compliance
# with the License. You may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing,
# software distributed under the License is distributed on an
# "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
# KIND, either express or implied. See the License for the
# specific language governing permissions and limitations
# under the License.

# Basic Airflow cluster configuration for CeleryExecutor with Redis and PostgreSQL.
#
# WARNING: This configuration is for local development. Do not use it in a
# production deployment.
#
# This configuration supports basic configuration using environment variables or
# an .env file
#
# The following variables are supported:
#
# AIRFLOW_IMAGE_NAME - Docker image name used to run Airflow.
#                      Default: apache/airflow:2.10.5
# AIRFLOW_UID - User ID in Airflow containers
#               Default: 50000
# AIRFLOW_PROJ_DIR - Base path to which all the files will be volumed.
#                   Default: .
# Those configurations are useful mostly in case of standalone testing/running
# Airflow in test/try-out mode
#
# _AIRFLOW_WWW_USER_USERNAME - Username for the administrator account (if
# requested).
#                               Default: airflow
# _AIRFLOW_WWW_USER_PASSWORD - Password for the administrator account (if
# requested).
#                               Default: airflow
# _PIP_ADDITIONAL_REQUIREMENTS - Additional PIP requirements to add when starting
# all containers.
#                               Use this option ONLY for quick checks. Installing
#                               requirements at container
#                               startup is done EVERY TIME the service is started.
#                               A better way is to build a custom image or extend
# the official image
#                               as described in
# https://airflow.apache.org/docs/docker-stack/build.html.
#                               Default: ''
#
# Feel free to modify this file to suit your needs.
---
x-airflow-common:
```

&airflow-common

```

# In order to add custom dependencies or upgrade provider packages you can use
your extended image.
# Comment the image line, place your Dockerfile in the directory where you placed
the docker-compose.yaml
# and uncomment the "build" line below, Then run `docker-compose build` to build
the images.
    image:      ${AIRFLOW_IMAGE_NAME:-apache/airflow:2.10.5}
    #          build:
    #          environment:
    #              &airflow-common-env
    AIRFLOW__CORE__EXECUTOR: CeleryExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CELERY__RESULT_BACKEND: db+postgresql://airflow:airflow@postgres/airflow
    AIRFLOW__CELERY__BROKER_URL:      redis://:@redis:6379/0
    AIRFLOW__CORE__FERNET_KEY:        ''
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
    AIRFLOW__CORE__LOAD_EXAMPLES:      'false'
    AIRFLOW__API__AUTH_BACKENDS:      'airflow.api.auth.backend.basic_auth,airflow.api.auth.backend.session'
    #          yamllint disable rule:line-length
    # Use simple http server on scheduler for health checks
    # See https://airflow.apache.org/docs/apache-airflow/stable/administration-and-deployment/logging-monitoring/check-health.html#scheduler-health-check-server
    #          yamllint enable rule:line-length
    AIRFLOW__SCHEDULER__ENABLE_HEALTH_CHECK: 'true'
    # WARNING: Use _PIP_ADDITIONAL_REQUIREMENTS option ONLY for a quick checks
    # for other purpose (development, test and especially production usage)
build/extend Airflow image.
    _PIP_ADDITIONAL_REQUIREMENTS: ${_PIP_ADDITIONAL_REQUIREMENTS:-duckdb}
    # The following line can be used to set a custom config file, stored in the
local config folder
    # If you want to use it, uncomment it and replace airflow.cfg with the name of
your config file
    #      AIRFLOW_CONFIG: '/opt/airflow/config/airflow.cfg'
    volumes:
    -      ${AIRFLOW_PROJ_DIR:-.}/dags:/opt/airflow/dags
    -      ${AIRFLOW_PROJ_DIR:-.}/logs:/opt/airflow/logs
    -      ${AIRFLOW_PROJ_DIR:-.}/config:/opt/airflow/config
    -      ${AIRFLOW_PROJ_DIR:-.}/plugins:/opt/airflow/plugins
    user:      "${AIRFLOW_UID:-50000}:0"
    depends_on:
        &airflow-common-depends-on
        redis:
            condition: service_healthy
        postgres:
            condition: service_healthy

services:
    #          #          postgres_dwh:
    #          image:      postgres:13
    #          environment:
    #              POSTGRES_USER:      postgres
    #              POSTGRES_PASSWORD: postgres
    #              POSTGRES_DB:      postgres
    #          restart:      always
    #          ports:
    #              -      "5432:5432"
    #          minio:
    image:      minio/minio:RELEASE.2025-02-18T16-25-55Z
    restart:      always
    volumes:
    -      ./data:/data
    environment:
    -      MINIO_ROOT_USER=minioadmin
    -      MINIO_ROOT_PASSWORD=minioadmin
    command:      server /data --console-address ":9001"
    ports:
    -      "9000:9000"
    -      "9001:9001"

```



```

command: <<: *airflow-common
          celery worker
          healthcheck:
#          yamllint disable rule:line-length
          test:
            - "CMD-SHELL"
            - 'celery --app airflow.providers.celery.executors.celery_executor.app
inspect ping -d "celery@${HOSTNAME}" || celery --app
airflow.executors.celery_executor.app inspect ping -d "celery@${HOSTNAME}"'
          interval: 30s
          timeout: 10s
          retries: 5
          start_period: 30s
          environment:
<<: *airflow-common-env
      # Required to handle warm shutdown of the celery workers properly
      # See https://airflow.apache.org/docs/docker-stack/entrypoint.html#signal-
propagation
          DUMB_INIT_SETSID: "0"
          restart: always
          depends_on:
<<: *airflow-common-depends-on
          airflow-init:
condition: service_completed_successfully

          airflow-triggerer:
<<: *airflow-common
          command: triggerer
          healthcheck:
test: ["CMD-SHELL", 'airflow jobs check --job-type TriggererJob --hostname "${
{HOSTNAME}}"']
          interval: 30s
          timeout: 10s
          retries: 5
          start_period: 30s
          restart: always
          depends_on:
<<: *airflow-common-depends-on
          airflow-init:
condition: service_completed_successfully

          airflow-init:
<<: *airflow-common
          entrypoint: /bin/bash
#          yamllint disable rule:line-length
          command:
            - -c
            |
            if [[ -z "${AIRFLOW_UID}" ]]; then
            echo
            echo -e "\033[1;33mWARNING!!!: AIRFLOW_UID not set!\e[0m"
            echo "If you are on Linux, you SHOULD follow the instructions below to set
"
            echo "AIRFLOW_UID environment variable, otherwise files will be owned by
root."
            echo "For other operating systems you can get rid of the warning with
manually created .env file:"
            echo "See:
https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/
index.html#setting-the-right-airflow-user"
            echo
            fi
            one_meg=1048576
            mem_available=$((($(getconf _PHYS_PAGES) * $(getconf PAGE_SIZE) /
one_meg))
            cpus_available=$(grep -cE 'cpu[0-9]+' /proc/stat)
            disk_available=$(df / | tail -1 | awk '{print $4}')
            warning_resources="false"
            if (( mem_available < 4000 )) ; then
            echo
            echo -e "\033[1;33mWARNING!!!: Not enough memory available for Docker.\

```

```

e[0m"
    echo "At least 4GB of memory required. You have $$((numfmt --to iec $(
((mem_available * one_meg)))"
    echo
    warning_resources="true"
    fi
    if (( cpus_available < 2 )); then
    echo
    echo -e "\033[1;33mWARNING!!!!: Not enough CPUS available for Docker.\e[0m"
    echo "At least 2 CPUs recommended. You have ${cpus_available}"
    echo
    warning_resources="true"
    fi
    if (( disk_available < one_meg * 10 )); then
    echo
    echo -e "\033[1;33mWARNING!!!!: Not enough Disk space available for
Docker.\e[0m"
    echo "At least 10 GBs recommended. You have $$((numfmt --to iec $(
((disk_available * 1024 )))"
    echo
    warning_resources="true"
    fi
    if [[ ${warning_resources} == "true" ]]; then
    echo
    echo -e "\033[1;33mWARNING!!!!: You have not enough resources to run
Airflow (see above)\e[0m"
    echo "Please follow the instructions to increase amount of resources
available:"
    echo
    https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/
index.html#before-you-begin"
    echo
    fi
    mkdir -p /sources/logs /sources/dags /sources/plugins
    chown -R "${AIRFLOW_UID}:0" /sources/{logs,dags,plugins}
    exec /entrypoint airflow version
    # yamllint enable rule:line-length
    environment:
        <<: *airflow-common-env
        _AIRFLOW_DB_MIGRATE: 'true'
        _AIRFLOW_WWW_USER_CREATE: 'true'
        _AIRFLOW_WWW_USER_USERNAME: ${_AIRFLOW_WWW_USER_USERNAME:-airflow}
        _AIRFLOW_WWW_USER_PASSWORD: ${_AIRFLOW_WWW_USER_PASSWORD:-airflow}
        _PIP_ADDITIONAL_REQUIREMENTS: ''
        user: "0:0"
    volumes:
        - ${AIRFLOW_PROJ_DIR:-.}:/sources

    airflow-cli:
        <<: *airflow-common
        profiles:
            - debug
        environment:
        <<: *airflow-common-env
        CONNECTION_CHECK_MAX_COUNT: "0"
    # Workaround for entrypoint issue. See:
    https://github.com/apache/airflow/issues/16252
    command:
        - bash
        - -c
        - airflow

    # You can enable flower by adding "--profile flower" option e.g. docker-compose --
profile flower up
    # or by explicitly targeted on the command line e.g. docker-compose up flower.
    # See: https://docs.docker.com/compose/profiles/
    flower:
        <<: *airflow-common
        command: celery
        profiles:
            - flower

```

```

                                ports:
                                - "5555:5555"
                                healthcheck:
test:  ["CMD", "curl", "--fail", "http://localhost:5555/"]
                                interval: 30s
                                timeout: 10s
                                retries: 5
                                start_period: 30s
                                restart: always
                                depends_on:
                                <<: *airflow-common-depends-on
                                airflow-init:
condition: service_completed_successfully

volumes:

                                postgres-db-volume:

```

ПРИЛОЖЕНИЕ В

Полный листинг req.txt и .gitignore

Файл req.txt фиксирует основные Python-зависимости. Файл .gitignore исключает локальные данные, логи, виртуальные окружения и потенциальные секреты из Git.

В.1 req.txt

```
apache-airflow==2.10.5
duckdb==1.2.2
```

В.2 .gitignore

```
# Byte-compiled / optimized / DLL files
__pycache__/
*.py[cod]
*$py.class

# C extensions
*.so

# Distribution / packaging
.Python
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
share/python-wheels/
*.egg-info/
.installed.cfg
*.egg
MANIFEST

# PyInstaller
# Usually these files are written by a python script from a template
# before PyInstaller builds the exe, so as to inject date/other infos into it.
*.manifest
*.spec

# Installer logs
pip-log.txt
pip-delete-this-directory.txt

# Unit test / coverage reports
htmlcov/
.tox/
.nox/
.coverage
.coverage.*
.cache
```

```

nosetests.xml
coverage.xml
*.cover
*.py.cover
.hypothesis/
.pytest_cache/
cover/

#
*.mo
*.pot

#
*.log
local_settings.py
db.sqlite3
db.sqlite3-journal

#
instance/
.webassets-cache

#
.scrappy

#
docs/_build/

#
.pybuilder/
target/

#
.ipynb_checkpoints

#
profile_default/
ipython_config.py

#
# For a library or package, you might want to ignore these files since the code is
# intended to run in multiple environments; otherwise, check them in:
# .python-version

#
# According to pypa/pipenv#598, it is recommended to include Pipfile.lock in
# version control.
# However, in case of collaboration, if having platform-specific dependencies or
# dependencies
# having no cross-platform support, pipenv may install dependencies that don't
work,
# install or all needed dependencies.
# Pipfile.lock

#
# Similar to Pipfile.lock, it is generally recommended to include uv.lock in
# version control.
# This is especially recommended for binary packages to ensure reproducibility,
and
# commonly is ignored for more
# libraries.
# uv.lock

#
# Similar to Pipfile.lock, it is generally recommended to include poetry.lock in
# version control.
# This is especially recommended for binary packages to ensure reproducibility,
and
# commonly is ignored for more
# https://python-poetry.org/docs/basic-usage/#commit-your-poetrylock-file-to-
version-control
# poetry.lock

```

```

# poetry.toml

# pdm
# Similar to Pipfile.lock, it is generally recommended to include pdm.lock in
version control.
# pdm recommends including project-wide configuration in pdm.toml, but
excluding .pdm-python.
# https://pdm-project.org/en/latest/usage/project/#working-with-version-control
# pdm.lock
# pdm.toml
.pdm-python
.pdm-build/

# pixi
# Similar to Pipfile.lock, it is generally recommended to include pixi.lock in
version control.
# pixi.lock
# Pixi creates a virtual environment in the .pixi directory, just like venv module
creates one
# in the .venv directory. It is recommended not to include this directory in
version control.
.pixi

# PEP 582; used by e.g. github.com/David-OConnor/pyflow and
github.com/pdm-project/pdm
__pypackages__/

# Celery stuff
celerybeat-schedule
celerybeat.pid

# Redis
*.rdb
*.aof
*.pid

# RabbitMQ
mnesia/
rabbitmq/
rabbitmq-data/

# ActiveMQ
activemq-data/

# SageMath parsed files
*.sage.py

# Environments
.env
.envrc
.venv
env/
venv/
ENV/
env.bak/
venv.bak/

# Spyder project settings
.spyderproject
.spyproject

# Rope project settings
.ropeproject

# mkdocs documentation
/site

# mypy
.mypy_cache/
.dmypy.json
dmypy.json

```

```

#                               Pyre                               type                               checker
.pyre/

#                               pytype                               static                               type                               analyzer
.pytype/

#                               Cython                               debug                               symbols
cython_debug/

#                               PyCharm
#   JetBrains specific template is maintained in a separate JetBrains.gitignore that
#   can
#                               be                               found                               at
https://github.com/github/gitignore/blob/main/Global/JetBrains.gitignore
#   and can be added to the global gitignore or merged into this file. For a more
#   nuclear
#   option (not recommended) you can uncomment the following to ignore the entire
#   idea                               folder.
#                               .idea/

#                               Abstra
#   Abstra is an AI-powered process automation framework.
#   Ignore directories containing user credentials, local state, and settings.
#   Learn more at https://abstra.io/docs
.abstra/

#                               Visual                               Studio                               Code
#   Visual Studio Code specific template is maintained in a separate
#   VisualStudioCode.gitignore
#   that can be found at
https://github.com/github/gitignore/blob/main/Global/VisualStudioCode.gitignore
#   and can be added to the global gitignore or merged into this file. However, if
#   you prefer,
#   you could uncomment the following to ignore the entire vscode folder
#                               .vscode/
#   Temporary file for partial code execution
tempCodeRunnerFile.py

#                               Ruff                               stuff:
.ruff_cache/

#                               PyPI                               configuration                               file
.pypirc

#                               Marimo
marimo/_static/
marimo/_lsp/
__marimo__/

#                               Streamlit
.streamlit/secrets.toml

data/
logs/
cred.py

```

ПРИЛОЖЕНИЕ Г

Скриншоты и чек-лист проверки работоспособности прототипа

Результаты практической проверки прототипа

1	Docker Compose поднял Airflow, PostgreSQL, Redis и MinIO	успешно
2	Airflow обнаружил DAG raw_from_api_to_s3 без ошибок импорта	успешно
3	Airflow Variables access_key и secret_key заданы для S3-доступа	успешно
4	DuckDB записал тестовый Parquet-файл в bucket bronze через S3 API	успешно
5	Запуск DAG дошёл до обращения к OpenSky API и ушёл в retry при таймауте	ожидаемая обработка сбоя

Вывод: локальная инфраструктура и запись в S3-совместимое хранилище подтверждены;
таймаут внешнего API показал работу retry-механизма Airflow и отражён как эксплуатационный риск.

Рисунок Г.1 – Сводка проверки инфраструктуры и DAG

Apache Airflow UI

DAGs

DAG: raw_from_api_to_s3 is_paused: False

Task	Operator	State	Description
start	EmptyOperator	success	Начальный маркер запуска
get_and_transfer_api_data_to_s3	PythonOperator	up_for_retry	Загрузка OpenSky API → MinIO/S3
end	EmptyOperator	none	Конечный маркер DAG

Результат тестового запуска

DAG обнаружен Airflow без import errors; задача стартовала и перешла в retry из-за таймаута внешнего OpenSky API.

Это подтверждает работоспособность оркестрации и демонстрирует штатную обработку временной недоступности API.

Секреты S3 настроены через Airflow Variables и не выводятся в отчёт.

Рисунок Г.2 – Экран Airflow с DAG raw_from_api_to_s3

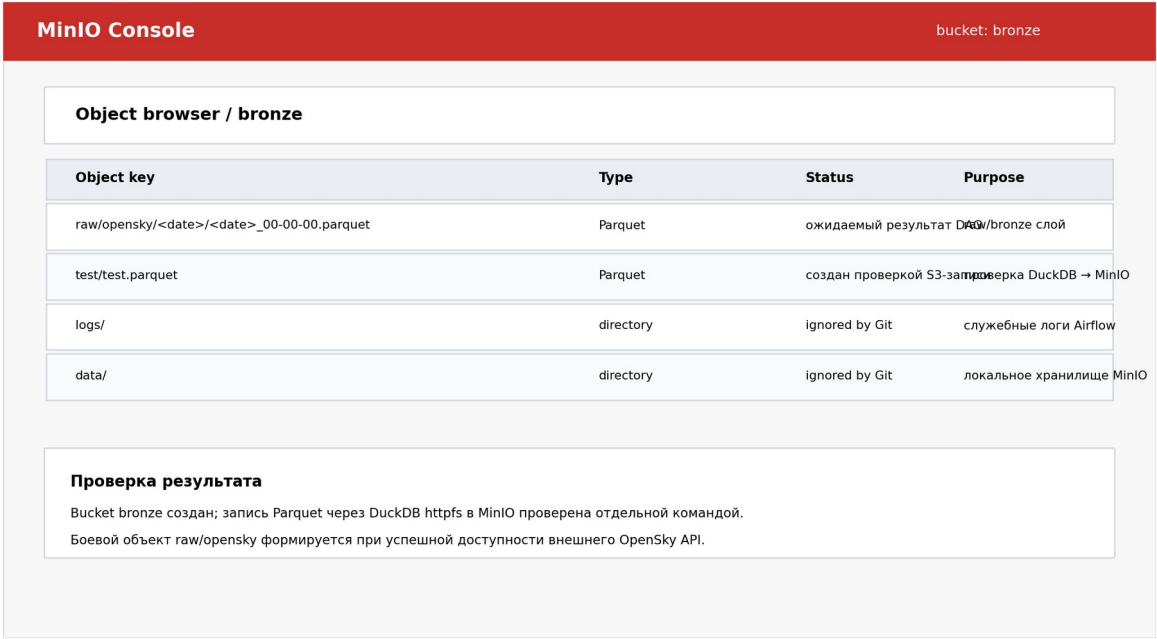


Рисунок Г.3 – Экран MinIO с bucket bronze

№	Проверка	Ожидаемый результат
1	Контейнеры Airflow, PostgreSQL, Redis и MinIO запущены через docker-compose.	Выполнено или подтверждено в ходе проверки.
2	В Airflow UI доступен DAG raw_from_api_to_s3.	Выполнено или подтверждено в ходе проверки.
3	В Airflow Variables заведены access_key и secret_key без публикации значений в Git.	Выполнено или подтверждено в ходе проверки.
4	В MinIO создан bucket bronze.	Выполнено или подтверждено в ходе проверки.
5	DuckDB успешно записывает тестовый Parquet-файл в MinIO через S3 API.	Выполнено или подтверждено в ходе проверки.
6	DAG применяет retry-политику при таймауте внешнего OpenSky API.	Выполнено или подтверждено в ходе проверки.
7	Реальные ключи доступа не включены в отчёт и репозиторий.	Выполнено или подтверждено в ходе проверки.